

Repetisjonsnotater

SOK-3023: Maskinlæring for økonomer

Fagansvarlig: Markus J. Aase

1. Klassifikasjon vs. Regresjon

Regresjon:

- ▶ Kontinuerlig/kvantitativ output.
- ▶ *Eks:* Boligpriser, inflasjon.

Klassifikasjon:

- ▶ Kategorisk/kvalitativ output.
- ▶ *Eks:* Svindel/ikke-svindel, godkjenne lån.

Notater

Hvorfor trenger vi Encoding?

- ▶ **Problem:** Maskinlæringsalgoritmer (som nevrale nettverk i TensorFlow) forstår kun *tall*, ikke tekst.
- ▶ **Utfordringen:** Mange datasett inneholder kategoriske variabler (f.eks. byer, farger, ukedager).
- ▶ **NB:** Å gi hver kategori et tall (Rød=1, Grønn=2, Blå=3) kan lure modellen til å tro at **Blå** er tre ganger så mye som Rød (kalt *Label Encoding*, kun bra for ordinale data).
- ▶ **'Riktig' løsning: One-Hot Encoding.** Vi lager en ny binær (0/1) kolonne for hver kategori.

Konsept: One-Hot Encoding

Rådata	
Farge	
Rød	
Grønn	
Blå	

One-Hot Encoding →

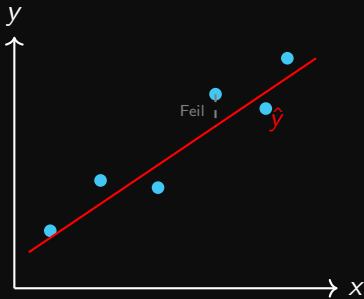
Hver kategori blir til sin egen dimensjon i rommet.

Kodet Data (Features)		
Er_Rød	Er_Grønn	Er_Blå
1	0	0
0	1	0
0	0	1

Fra teori til praksis: Lineær regresjon

1. Det matematiske problemet

Vi ønsker å finne linjen $\hat{y} = \beta_0 + \beta_1 x$ som minimerer feilen (avstanden mellom punktene og linjen).



2. Løsningen i Python

I `scikit-learn` skjules OLS-beregninger. Alt vi trenger er 3 logiske steg:

`regresjon.py`

```
from sklearn.linear_model import
LinearRegression

model = LinearRegression()

# 2. Trener modellen
model.fit(X_train, y_train)

# 3. Prediksjon på nye data
y_pred = model.predict(X_test)
```

Evaluering av klassifikasjon vs regresjon

Klassifikasjon

Nøyaktighet (Accuracy):

$$\text{Acc} = \frac{TP + TN}{TP + TN + FP + FN}$$

Presisjon (Precision):

$$P = \frac{TP}{TP + FP}$$

Sensitivitet (Recall):

$$R = \frac{TP}{TP + FN}$$

Regresjon

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

R-squared (R^2):

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

2. Kompleksitet & Ikke-linearitet i økonomien

- ▶ Økonomi bruker ofte lineære forenklinger (f.eks. tradisjonell OLS).
- ▶ **Universal Approximation Theorem:** Et nevralt nettverk kan tilnærme *enhver* kontinuerlig funksjon.
- ▶ **Nytten:** Lar oss fange opp komplekse samspill uten å gjette *matematisk form* på forhånd.

Notater

Prisen vi betaler: Black-Boxproblemet

Nøyaktighet vs. Tolkbarhet

- ▶ **Tradisjonell OLS:** Gir klare marginaleffekter (β). Vi kan forklare nøyaktig *hvordan* X påvirker Y .
- ▶ **Black-Box (NN):** Modellen fanger opp komplekse mønstre...
- ▶ Kan vi stole på en modell vi ikke forstår? Hvordan forklarer vi et avslag på lån til kunden?
- ▶ **Løsninger (XAI):** Metoder som *SHAP-verdier* brukes nå i økonomien for å 'titte inn' i den sorte boksen.

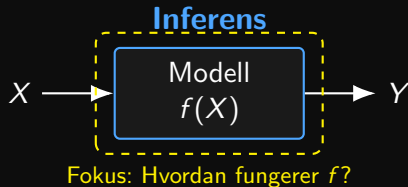
Notater

Prediksjon vs. Inferens

Hva er egentlig målet vårt?

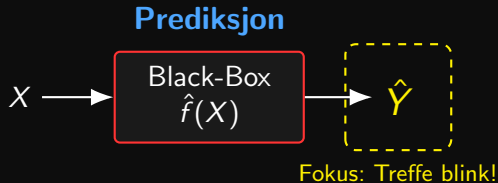
► Inferens (Økonometri):

- *Mål:* Forstå **sammenhengen**.
- *Fokus:* Finne kausalitet, estimere sanne parametere (β), og teste hypoteser.
- *Krav:* Modellen må være tolkbar.



► Prediksjon (Maskinlæring):

- *Mål:* Gjette **utfallet** for nye data.
- *Fokus:* Minimere feilen mellom Y og vår prediksjon $\hat{Y}$.
- *Krav:* Kan gjerne være en 'black-box', så lenge prediksjonene er bra!



Hva er en tensor?

Tensor In Tensorflow

Tensors are multi-dimensional arrays used to represent data in TensorFlow

Scaler

1

Vector

$$\begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

Matrix

$$\begin{bmatrix} 7 & 2 \\ 8 & 4 \end{bmatrix}$$

Tensor

$$\begin{bmatrix} \begin{bmatrix} 7 & 2 \end{bmatrix} & \begin{bmatrix} 2 & 7 \end{bmatrix} \\ \begin{bmatrix} 8 & 4 \end{bmatrix} & \begin{bmatrix} 4 & 8 \end{bmatrix} \end{bmatrix}$$

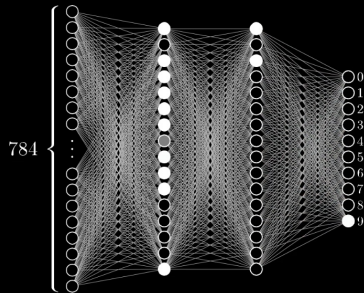
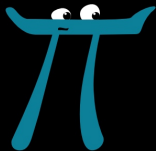
3. CNN vs. Tradisjonelle Nettverk (MLP)

- ▶ **MLP (Vanilla NN):** *Flater ut* data. Mister den romlige strukturen (pikslar ved siden av kvarandre).
- ▶ **CNN:** Bevarer strukturen (tensorer).
- ▶ **Konvolusjon:** Filtre/kernels glir over dataene for å finne lokale mønstre.
- ▶ **Pooling:** Abstraherer og nedskalrer (down-sampling).

Notater

Nevrale nettverk

Does the network
actually do this?



Pixels



Edges

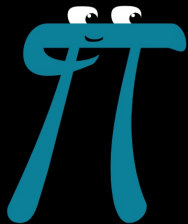


Shapes



Digits

Vi har gjort dette på to ulike vis

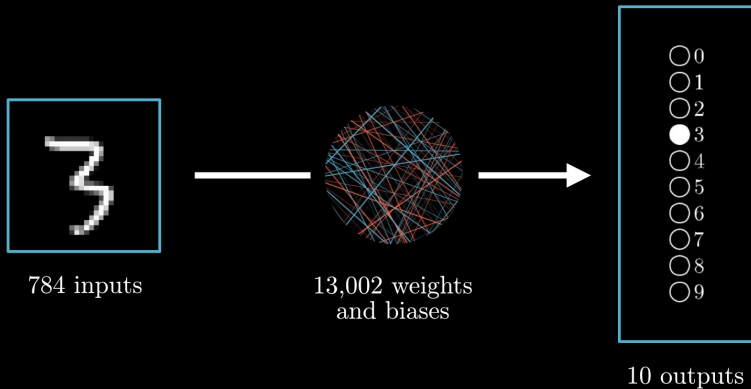


'Øvelse' gjør mester

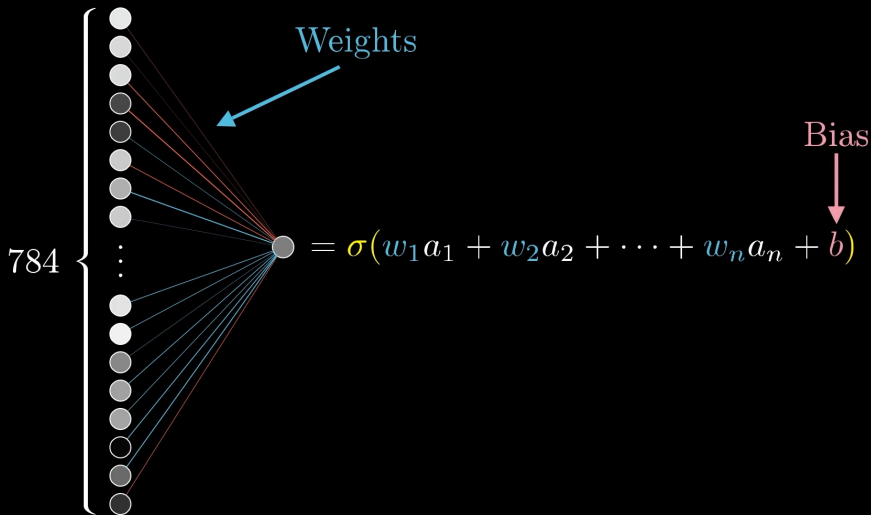
(0, 0) (6, 6) (3, 3) (6, 6) (7, 7) (8, 8) (0, 0) (9, 9)
(5, 5) (4, 4) (3, 3) (6, 6) (5, 5) (8, 8) (9, 9) (5, 5)
(4, 4) (4, 4) (7, 7) (2, 2) (0, 0) (3, 3) (2, 2) (8, 8)
(9, 9) (1, 1) (9, 9) (2, 2) (2, 2) (7, 7) (9, 9) (4, 4)
(8, 8) (7, 7) (4, 4) (1, 1) (3, 3) (1, 1) (5, 5) (3, 3)
(2, 2) (3, 3) (9, 9) (0, 0) (9, 9) (9, 9) (1, 1) (5, 5)
(8, 8) (4, 4) (1, 7) (7, 7) (4, 4) (4, 4) (4, 4) (2, 2)
(0, 0) (7, 7) (2, 2) (4, 4) (8, 8) (2, 2) (6, 6) (9, 9)
(9, 9) (2, 2) (8, 8) (7, 7) (6, 6) (1, 1) (1, 1) (2, 2)
(3, 3) (9, 9) (1, 1) (6, 6) (5, 5) (1, 1) (1, 1) (0, 0)

Hva gjør vi?

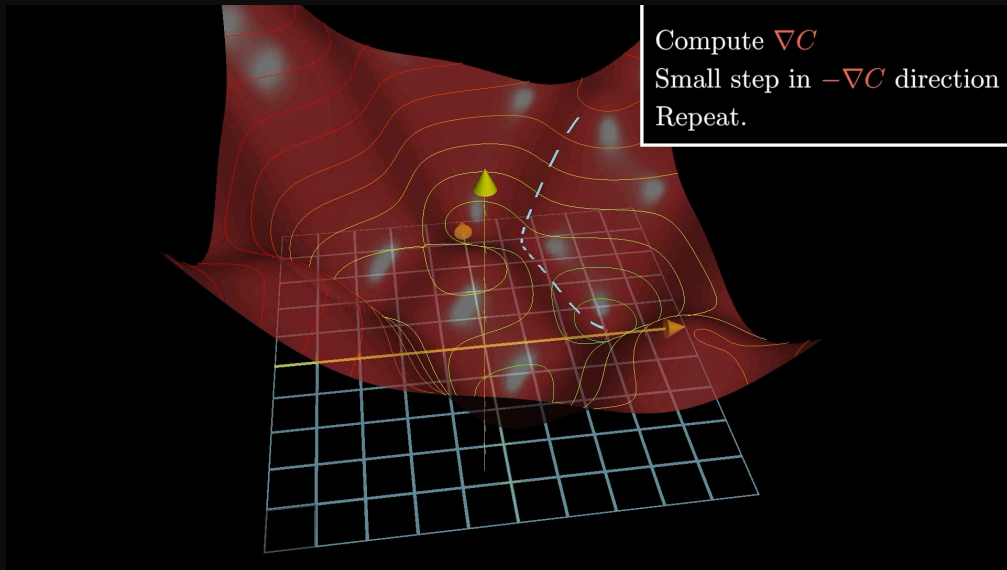
Neural network function



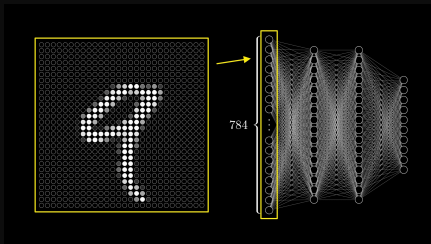
Vektor, biaser og sånt



Minimere tapsfunksjon!

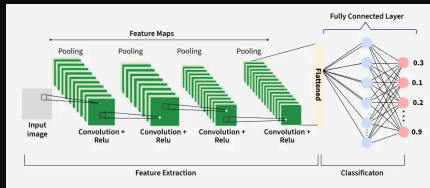


MLP (Tradisjonelt nettverk)



- ▶ **Input:** Flater utdataene.
- ▶ **Romlig struktur:** Mister informasjon om pikslenes plassering i forhold til hverandre.
- ▶ **Parametere:** Fullt tilkoblet (dense). Fører til enormt mange vekter for store bilder.
- ▶ **Beste bruk:** Strukturerte data

CNN (Konvolusjonelt nettverk)

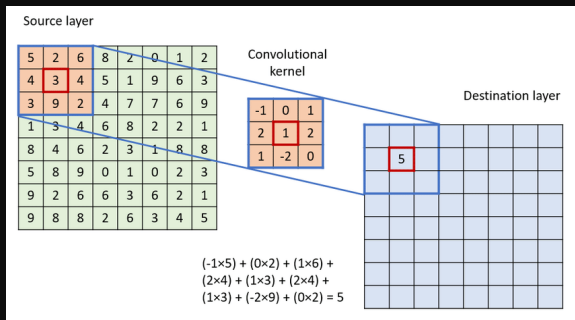


- ▶ **Input:** Bevarer dataenes flerdimensjonale form (tensorer).
- ▶ **Romlig struktur:** Bruker glidende *filtre/kernels* for å fange opp lokale mønstre (kanter, former).
- ▶ **Parametere:** Deler på vektene (parameter sharing), noe som gjør modellen mye mer effektiv.
- ▶ Utviklet for visuelle data som bilder og video.

Kjernen i bildeanalyse

- ▶ **Konvolusjon:** En matematisk operasjon der vi glir en liten matrise over et bilde for å transformere dataene.
- ▶ **Kernel / Filter:** Selve matrisen (f.eks. 3×3 piksler) som leter etter spesifikke mønstre som kanter, hjørner eller fargeoverganger.
- ▶ **Feature Extraction:** Målet med konvolusjon. Nettverket lærer se/v hvilke visuelle trekk som er viktigst for å løse oppgaven.
- ▶ **Feature Map:** Resultatet vi får etter at filteret har prosessert bildet – et kart over hvor et bestemt trekk befinner seg.

Visuell forklaring



Pooling: Nedskalere mengden informasjonen

Hvorfor bruke pooling?

- ▶ Etter konvolusjon har vi funnet viktige trekk (features), men vi har ofte veldig mye data.
- ▶ **Mål med pooling:** Nedskalering (down-sampling).
- ▶ Vi reduserer dimensjonene (og antall parametere), noe som gjør beregningene raskere og reduserer faren for overtilpasning (overfitting).
- ▶ Den gjør også modellen mer robust for små forskyvninger (translation invariance).

Begrepet "Stride" (Skrittlengde)

- ▶ **Stride** bestemmer hvor mange piksler vi flytter filteret/pooling-vinduet for hvert steg.
- ▶ En *stride* på 1 betyr at vi glir vinduet én piksel av gangen (overlapping).
- ▶ En *stride* på 2 (ofte brukt med et 2×2 vindu) gjør at vi ikke overlapper, og halverer dermed høyde og bredde.

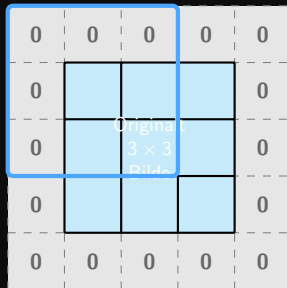
De tre hyperparameterne for CNN/Pooling

Styring av nettverkets flyt

- **Filterstørrelse (Kernel size):** Hvor stort er vinduet som glir over bildet? (f.eks. 3×3 eller 2×2). Et større filter ser mer av helheten, mens et mindre filter fanger fine detaljer.
- **Stride (Skrittlengde):** Hvor mange piksler vi flytter vinduet for hvert steg. Stride = 1 gir overlapp og bevarer størrelsen godt. Stride = 2 halverer gjerne bildestørrelsen.
- **Padding (Fyll):** Å legge til en ramme av ekstra piksler (vanligvis nuller, kalt "Zero Padding") rundt originalbildet før filteret brukes.

Illustrasjon av Zero Padding

Filter (3×3) plassert på kanten



Max Pooling vs. Mean Pooling (Stride = 2)

Max Pooling

- ▶ Henter ut den **høyeste** verdien i vinduet.
- ▶ Fanger opp de mest markante trekkene (f.eks. kanter). Mest brukt i praksis.

1	3	0	4
2	8	6	2
3	1	0	1
1	5	7	2

→ Max

8	6
5	7

Mean (Average) Pooling

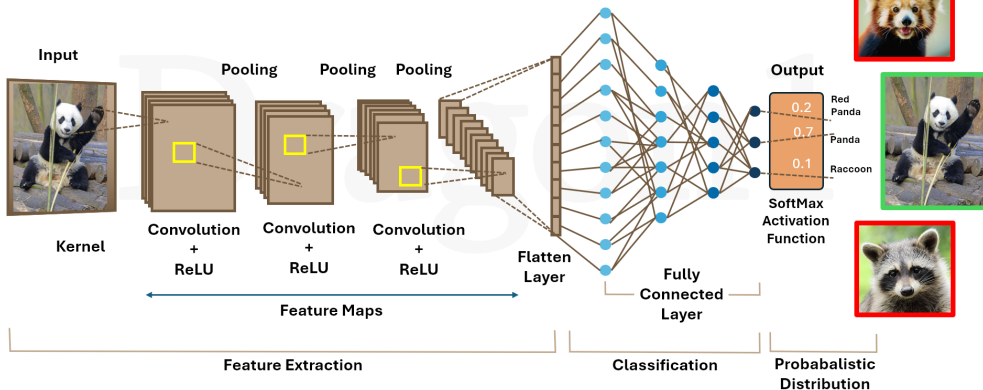
- ▶ Regner ut **gjennomsnittet** av verdiene i vinduet.
- ▶ Glatte ut bildet, klemmersammen informasjonen mer jevnt.

1	3	0	4
2	8	6	2
3	1	0	1
1	5	7	2

→ Mean

3.5	3
2.5	2.5

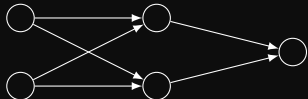
Convolutional Neural Networks (AI Deep Learning)



1. Feedforward Networks vs Recurrent Networks

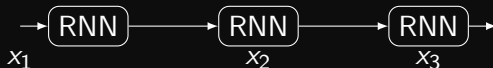
Feedforward Neural Network

- ▶ Input går én vei gjennom nettverket.
- ▶ Ingen hukommelse.
- ▶ Hvert input behandles uavhengig.
- ▶ Passer for:
 - ▶ Bilder
 - ▶ Klassifikasjon
 - ▶ Statisk data

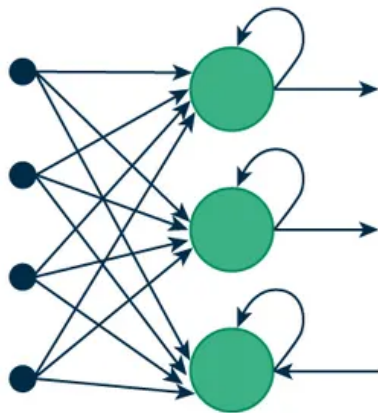


Recurrent Neural Network (RNN)

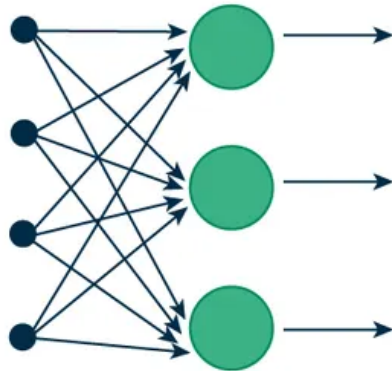
- ▶ Har intern hukommelse.
- ▶ Tidligere tilstander påvirker fremtiden.
- ▶ Samme vektor brukes ved hvert tidspunkt.
- ▶ Passer for:
 - ▶ Tekst
 - ▶ Tale
 - ▶ Tidsserier
 - ▶ Sekvenser



Eksempel



(a) Recurrent Neural Network



(b) Feed-Forward Neural Network

2. Hvordan fungerer en RNN?

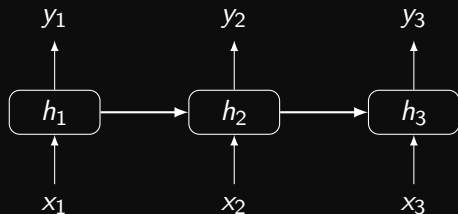
Idé: Nettverket bygger opp en skjult tilstand over tid.

Ved tidspunkt t :

$$h_t = \sigma(W_x x_t + W_h h_{t-1} + b_h)$$

- ▶ x_t = input nå
- ▶ h_{t-1} = hukommelse fra tidligere
- ▶ h_t = ny skjult tilstand

RNN-en prøver å komprimere hele historien inn i h_t .



3. Vanishing Gradient Problem

Problem: Gradientene blir mindre og mindre bakover i tid.

Ved backpropagation multipliseres mange deriverte:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=1}^k W_h^\top \phi'(z_i)$$

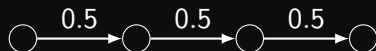
Hvis:

$$|W_h \phi'(z)| < 1$$

så krymper gradienten eksponentielt.

Konsekvens:

- ▶ Tidlig informasjon glemmes



$$0.5^4 = 0.0625$$

Gradienten blir nesten null etter mange steg.

4. Exploding Gradient Problem

Motsatt problem: Gradientene kan eksplodere.

Hvis:

$$|W_h \phi'(z)| > 1$$

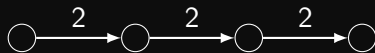
så vokser gradientene eksponentielt.

$$2^6 = 64$$

Konsekvenser:

- ▶ Ustabil trening
- ▶ Ekstreme parameteroppdateringer
- ▶ Numerisk overflow / NaN

Vanlig løsning:



$$2^4 = 16$$

Gradienten blir enorm etter mange steg.

5. LSTM – Long Short-Term Memory

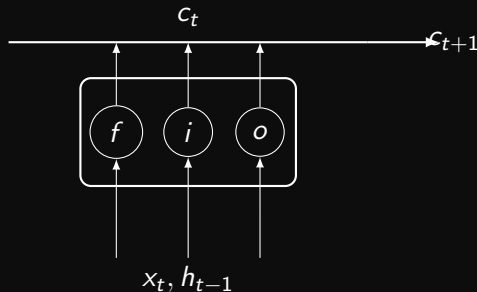
LSTM løser hukommelsesproblemet ved å introdusere:

► En **cell state**:

c_t

- Informasjon kan flyte nesten lineært gjennom tid.
- Porter (*gates*) styrer informasjonen:
 - **Forget gate**: Hva skal glemmes?
 - **Input gate**: Hva skal lagres?
 - **Output gate**: Hva sendes videre?

Gradienten får en “motorvei” bakover gjennom tid.



LSTM Cell

LSTM = Long Short-Term Memory

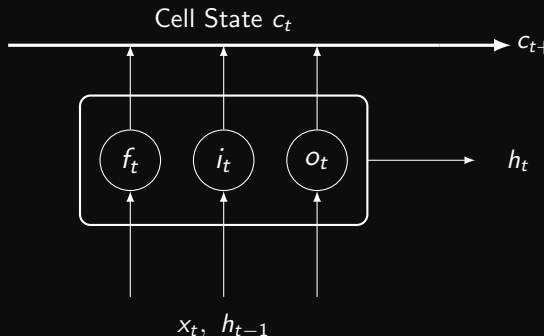
En vanlig RNN mister ofte informasjon over lange sekvenser. LSTM introduserer derfor:

- ▶ en **cell state** c_t
- ▶ flere **gates** som kontrollerer informasjon

$$f_t = \sigma(W_f[h_{t-1}, x_t])$$

$$i_t = \sigma(W_i[h_{t-1}, x_t])$$

$$o_t = \sigma(W_o[h_{t-1}, x_t])$$



Hovedidé: Cell state fungerer slik at vi prøver å huske det vi må, og glemme det unødvendige.

4. LSTM — 'Forsøk på oppsummering'

- ▶ **Vanishing Gradient:** I vanlige RNN krymper gradientene bakover gjennom tid. Modellen 'glemmer' hvor den startet.
- ▶ **LSTM:** *Long Short-Term Memory* introduserer en cellestatus (cell state).
- ▶ **Porter (Gates):** Styrer informasjonen:
 - ▶ *Forget:* Hva skal vi kaste?
 - ▶ *Input:* Hva er nytt og viktig?
 - ▶ *Output:* Hva skal vi sende videre?

Notater

Takk til 3Blue1Brown

Flere av de visuelle konseptene og figurene brukt i disse notatene er sterkt inspirert av (eller lånt fra) YouTube-kanalen **3Blue1Brown**.

- ▶ En fantastisk ressurs for visuell matematikk.
- ▶ Anbefales på det sterkeste for å bygge en dypere, intuitiv forståelse av lineær algebra og nevrale nettverk!